

Dokumentacja Bazy Danych - ASRPweb

ASRPweb, czyli **Akademicki System Rezerwacji Przedmiotów** to nowoczesna platforma służąca przede wszystkim studentom do rejestrowania się do zajęć prowadzonych na danej uczelni. Z Systemu korzystają również prowadzący, którzy mogą na bieżąco dodawać, usuwać czy modyfikować prowadzone przez nich zajęcia. Zarówno studenci jak i prowadzący mają wgląd do całego swojego planu, szczegółowych informacji o każdym prowadzonym przedmiocie na uczelni, a także dostęp do indywidualnej historii aktywności na platformie.

Niniejsza dokumentacja szczegółowo opisuje architekturę, strukturę tabel, zasady spójności danych, realizację operacji CRUD, transakcyjnych i raportujących w bazie danych systemu ASRPweb, a także role, kontrolę równoczesnego dostępu i uprawnień do bazy danych.

Autorami projektu są **Szymon Kuźba** i **Kacper Mucha**.

1. Wykorzystane Technologie

Projekt został zrealizowany z podziałem na warstwę serwerową (backend) oraz interfejs użytkownika (frontend).

Backend:

- **Python 3** – Główny język logiki serwerowej.
- **FastAPI** – Nowoczesny, asynchroniczny framework do budowy interfejsów API, gwarantujący wysoką wydajność oraz automatyczną generację dokumentacji (Swagger UI).
- **SQLAlchemy** – Zaawansowany system ORM (Object-Relational Mapping), pozwalający na mapowanie klas Pythona na tabele relacyjnej bazy danych i zapewniający bezpieczeństwo przed atakami SQL Injection.
- **Pydantic** – Biblioteka do rygorystycznej walidacji danych wejściowych i wyjściowych (np. walidacja żądań zapisu na zajęcia).
- **Uvicorn** – Serwer ASGI używany do hostowania aplikacji FastAPI.
- **PostgreSQL / SQLite** – Relacyjny silnik bazy danych (projekt wspiera oba rozwiązania dzięki abstrakcji ORM).

Frontend:

- **HTML5, CSS3** – Tworzenie struktury i wyglądu strony internetowej symulującej interfejs systemu ASRPweb.
 - **Vanilla JavaScript** – Logika frontendu bez użycia ciężkich frameworków. Wykorzystuje `Fetch API` do asynchronicznej komunikacji z backendem oraz `localStorage` do przechowywania stanu aplikacji (np. wybranej roli symulowanego użytkownika).
-

2. Model Relacyjny i Schema (Models)

Baza danych składa się z 5 głównych tabel realizujących relacje studentów, wykładowców, zajęć, rejestracji oraz audytu.

teachers (Prowadzący)

Przechowuje dane kadry akademickiej prowadzącej zajęcia.

- `id` (Integer, Primary Key) – Unikalny identyfikator.
- `first_name` (String(50)) – Imię.
- `last_name` (String(50)) – Nazwisko.
- `academic_title` (String(20)) – Tytuł naukowy (np. *dr*, *prof.*).

students (Studenci)

Przechowuje dane studentów uczelni.

- `id` (Integer, Primary Key) – Unikalny identyfikator.
- `first_name` (String(50)) – Imię.
- `last_name` (String(50)) – Nazwisko.
- `index_number` (String(10), Unique) – Unikalny numer indeksu studenta.

classes (Zajęcia)

Reprezentuje konkretne przedmioty prowadzone na uczelni.

- `id` (Integer, Primary Key) – Unikalny identyfikator przedmiotu.
- `name` (String(100)) – Nazwa przedmiotu.
- `max_seats` (Integer) – Limit miejsc na zajęciach.
- `taken_seats` (Integer) – Liczba aktualnie zajętych miejsc.
- `teacher_id` (Integer, Foreign Key `teachers.id`) – ID prowadzącego zajęcia.
- `day_of_week` (Integer) – Dzień tygodnia (1 = Poniedziałek, ..., 5 = Piątek).
- `start_time` (String(5)) – Godzina rozpoczęcia w formacie `HH:MM`.

- `end_time` (String(5)) – Godzina zakończenia w formacie HH:MM .
- `room` (String(20)) – Numer sali.
- `week_parity` (Integer) – Parzystość tygodnia (0 - co tydzień, 1 = TP - parzysty, 2 = TN - nieparzysty).

enrollments (Zapisy)

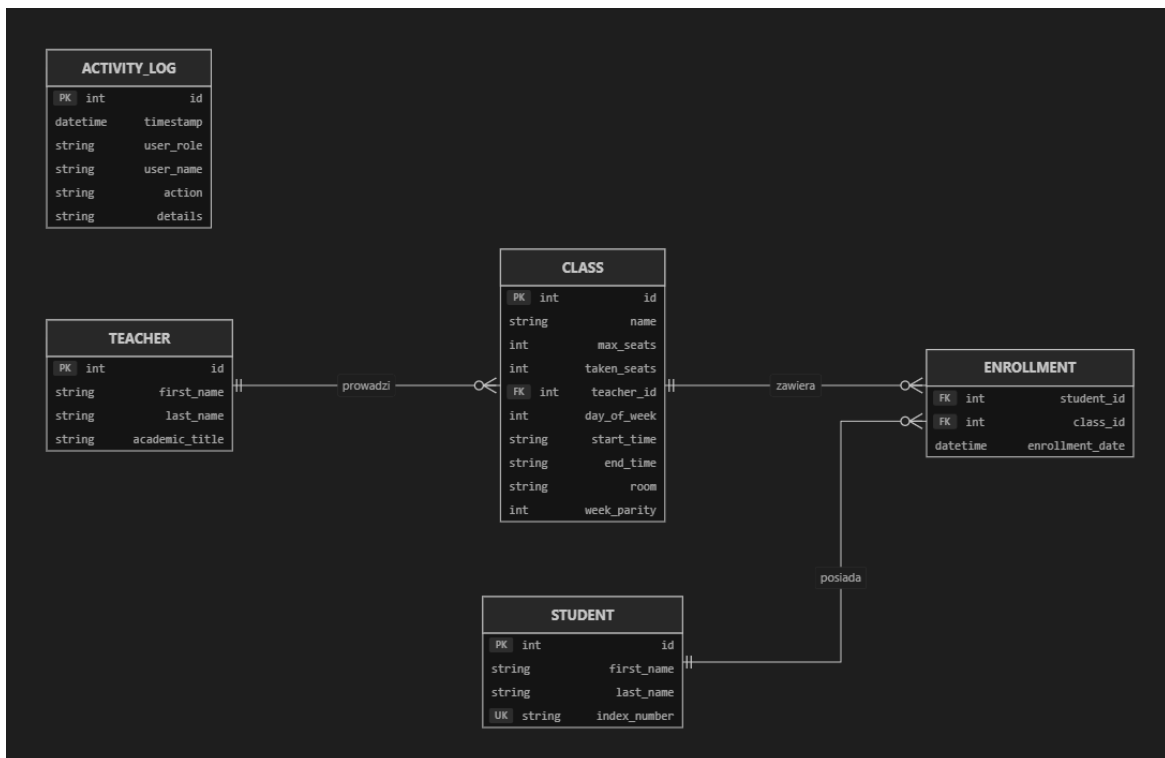
Tabela łącząca, reprezentująca relację wiele-do-wielu (M:N) pomiędzy studentami a zajęciami.

- `student_id` (Integer, Foreign Key `students.id` , Primary Key) – ID studenta.
- `class_id` (Integer, Foreign Key `classes.id` , Primary Key) – ID zajęć.
- `enrollment_date` (DateTime) – Dokładny stempel czasowy zapisu (czas polski UTC+2).

activity_logs (Dziennik zdarzeń / Audit Trail)

Autonomiczna tabela audytowa zbierająca historię operacji użytkowników w systemie.

- `id` (Integer, Primary Key) – ID wpisu.
- `timestamp` (DateTime) – Data i czas wykonania operacji (czas polski UTC+2).
- `user_role` (String(20)) – Rola wykonawcy (`student` , `teacher` , `system`).
- `user_name` (String(100)) – Dane wykonawcy.
- `action` (String(50)) – Typ zdarzenia (`ENROLL` , `UNENROLL` , `CREATE_CLASS` , `EDIT_CLASS` , `DELETE_CLASS` , `RESET_DB`).
- `details` (String(500)) – Szczegółowy opis wykonanej czynności.



3. Realizacja Operacji w Bazie Danych

A. Transakcyjny Zapis Studenta z Blokowaniem Pesymistycznym (FOR UPDATE)

W celu zapobiegania zjawisku wyścigu (race condition) – np. gdy kilku studentów próbuje zapisać się na ostatnie wolne miejsce w ułamku sekundy – zastosowano mechanizm blokowania pesymistycznego w bazie danych.

W bazie realizowane jest to za pomocą klauzuli `FOR UPDATE` poprzez SQLAlchemy:

```
course = db.query(models.Class). \
    filter(models.Class.id == request.class_id). \
    with_for_update(). \
    first()
```

Przebieg operacji:

1. Sesja bazy danych zakłada blokadę na wybranym wierszu tabeli `classes`.
2. Pobierany jest aktualny stan liczby zajętych miejsc `taken_seats`.
3. Następuje walidacja `taken_seats < max_seats`.

4. Jeśli limit nie został przekroczony, dodawany jest rekord w tabeli `enrollments`, inkrementowane jest pole `taken_seats` o 1 w tabeli `classes` i następuje zatwierdzenie transakcji (`db.commit()`), co zwalnia blokadę.
5. Inne równoległe zapytania na ten sam rekord oczekują, aż pierwsza transakcja zostanie zatwierdzona lub wycofana.

B. Kaskadowe Usuwanie Zajęć

Podczas usuwania przedmiotu przez prowadzącego, próba usunięcia rekordu z tabeli `classes` bezpośrednio wywołałaby błąd naruszenia więzów integralności klucza obcego (Foreign Key Constraint Failure), ponieważ w tabeli `enrollments` nadal znajdowałiby się studenci zapisani na ten przedmiot.

Operację zrealizowano jako transakcję usuwającą najpierw powiązania zależne:

```
# 1. Usunięcie wszystkich studentów przypisanych do tych zajęć z tabeli łączącej
db.query(models.Enrollment).filter(models.Enrollment.class_id == class_id).delete()

# 2. Bezpieczne usunięcie głównych zajęć
db.delete(course)

# 3. Zapisanie operacji w dzienniku audytu i zatwierdzenie zmian
db.commit()
```

C. Walidacja Konfliktów Czasowych i Parzystości Tygodni (Collision Checking)

System automatycznie chroni przed nałożeniem się zajęć w planie.

Walidacja dla Prowadzącego:

Sprawdza, czy nauczyciel nie prowadzi już innych zajęć w tym samym oknie czasowym:

```
# Algorytm nakładania się przedziałów (A_start < B_end oraz B_start < A_end)
query = db.query(models.Class).filter(
    models.Class.teacher_id == teacher_id,
    models.Class.day_of_week == day_of_week
)

# Jeśli badamy edycję, wykluczamy aktualnie modyfikowane zajęcia z wyników
```

```
if exclude_class_id:
    query = query.filter(models.Class.id != exclude_class_id)
```

Dla każdego z zajęć w wybranym dniu sprawdzane jest:

1. **Parzystość tygodnia:** Kolizja zachodzi, gdy nowe zajęcia lub istniejące odbywają się "co tydzień" (`week_parity == 0`) LUB parzystości są identyczne (`week_parity == existing.week_parity`).
2. **Czas trwania:** Konwersja czasu na minuty od początku dnia:

```
def to_minutes(t_str):
    h, m = map(int, t_str.split(':'))
    return h * 60 + m
```

Kolizja występuje, jeśli `new_start < existing_end` oraz `existing_start < new_end` .

Walidacja dla Studenta (Zapisy):

Podobny algorytm sprawdza obecny plan zajęć studenta. Weryfikowane są godziny i parzystości wszystkich przedmiotów, na które student jest już zapisany, zapobiegając dodaniu zajęć kolidujących.

D. Realizacja operacji CRUD na tabeli i autoryzacja

System umożliwia prowadzącym pełne zarządzanie przedmiotami z tabeli `classes` (tworzenie, odczyt, aktualizacja, usuwanie – CRUD). Aby zapobiec nieautoryzowanym modyfikacjom, oprócz fizycznych zmian na frontendzie, zaimplementowano mechanizm kontroli dostępu opartej na właścicielu rekordu (Ownership-based Access Control).

Przed wykonaniem operacji `UPDATE` (edycja) lub `DELETE` (usunięcie), silnik bazodanowy pobiera dany rekord i weryfikuje, czy identyfikator prowadzącego zgłaszającego żądanie pokrywa się z kluczem obcym przypisanym do zajęć. W przypadku braku zgodności, transakcja jest natychmiast przerywana, a system zwraca błąd `403 Forbidden` :

```
course = db.query(models.Class).filter(models.Class.id == class_id).first()

# Weryfikacja właściciela (Tarcza ochronna)
if course.teacher_id != request.teacher_id:
    raise HTTPException(
```

```
status_code=403,  
detail="Brak uprawnień. Możesz modyfikować wyłącznie swoje zajęcia!"  
)
```

Zastosowanie: Nawet jeśli użytkownik ominie zabezpieczenia nawigacyjne w JavaScript (np. wymuszając żądanie HTTP z poziomu konsoli), serwer Pythona każdorazowo przed operacjami zapisu/usuwania (CRUD) dokonuje weryfikacji tożsamości i własności rekordu. Wykorzystanie RBAC (Role-Based Access Control) z weryfikacją na poziomie ORM stanowi żelazną barierę bezpieczeństwa.

E. Operacja o charakterze raportującym z kilku tabel

System realizuje złożone operacje odczytu (raportujące), które dostarczają użytkownikom oraz administratorom pełny obraz stanu uczelni bez modyfikowania jej stanu.

Sztandarowym przykładem jest generowanie kompletnego katalogu zajęć (tzw. "Karty Zajęć"), co wymaga jednoczesnego pobrania, złączenia i zagregowania danych z aż czterech różnych encji: `classes`, `teachers`, `students` oraz `enrollments`.

Zamiast wykonywać wiele trywialnych zapytań dla pojedynczych elementów, system wykorzystuje możliwości ORM do wykonania złączeń (`JOIN`) i budowy złożonego zapytania. W endpointzie `/api/classes/details` realizowany jest dynamiczny raport list obecności dla wszystkich przedmiotów:

```
classes = db.query(models.Class).order_by(models.Class.id).all()  
result = []  
for c in classes: # zagnieżdżenie - dla każdego przedmiotu...  
    teacher = db.query(models.Teacher).filter(models.Teacher.id == c.teacher_id).first()  
    teacher_name = f"{teacher.academic_title or ''} {teacher.first_name} \\  
        {teacher.last_name}".strip() if teacher else "Nieznany"  
  
    students = db.query(models.Student).join(  
        models.Enrollment, models.Enrollment.student_id == models.Student.id  
    ).filter(models.Enrollment.class_id == c.id).order_by(  
        models.Student.last_name, models.Student.first_name).all()  
  
    student_list = []  
    for s in students: # kolejne zagnieżdżenie - wszyscy studenci  
        student_list.append({  
            "id": s.id,  
            "first_name": s.first_name,  
            "last_name": s.last_name,
```

```
"index_number": s.index_number
})
```

Złączenia relacyjne (INNER JOIN): Wykorzystanie tabeli asocjacyjnej enrollments do rozwiązania relacji wiele-do-wielu (M:N) pomiędzy studentami a zajęciami, filtrując wyniki wyłącznie do poszukiwanego przedmiotu.

Sortowanie na poziomie silnika SQL: Zlecenie bazie danych uporządkowania wyników (klauszula ORDER BY) odciąża pamięć podręczną serwera aplikacyjnego.

Obiektowa agregacja danych (Data Shaping): Baza relacyjna zwraca dane w postaci płaskiej (tabelarycznej). Serwer FastAPI agreguje te wyniki i dokonuje transformacji do postaci hierarchicznej (struktury JSON). Płaskie rekordy są mapowane w jeden główny obiekt "Zajęć", który zawiera w sobie właściwości nauczyciela oraz zagnieżdżoną wewnątrz tablicę (listę) z obiektami przypisanych do nich studentów.

Zastosowanie: Tak przygotowany, złożony dokument wysyłany jest jako pojedyncza odpowiedź HTTP, co minimalizuje opóźnienia sieciowe i pozwala interfejsowi użytkownika (frontendowi) na natychmiastowe wygenerowanie interaktywnego, zagregowanego widoku wszystkich zajęć.

F. Automatyczne Logowanie Działań (Audit Trail)

Po pomyślnym zatwierdzeniu jakiegokolwiek modyfikującej operacji w bazie danych wywoływana jest funkcja `log_activity()`, tworząca nowy wpis audytowy:

```
def log_activity(db: Session, user_role: str, user_name: str, action: str, details: str):
    log_entry = models.ActivityLog(
        user_role=user_role,
        user_name=user_name,
        action=action,
        details=details
    )
    db.add(log_entry)
    db.commit()
```

Wpisy te nie są powiązane kluczami obcymi z tabelami `students` i `teachers`, dzięki czemu usunięcie konta użytkownika nie powoduje utraty logów historycznych (zachowana integralność dowodowa).

G. Inicjalizacja i Resetowanie Sekwencji Auto-increment (PostgreSQL Sequence Reset)

Podczas operacji seedowania lub resetowania bazy danych do stanu fabrycznego (`db.reset-db`), po usunięciu i ponownym wstawieniu startowych rekordów z wymuszonymi identyfikatorami (`id=1` , `id=2` itd.), wewnętrzne liczniki sekwencji PostgreSQL mogłyby wskazywać na wartość początkową `1` , co skutkowałoby błędem `UniqueViolation` przy pierwszej próbie dodania nowego rekordu przez użytkownika.

Baza danych ASRPweb zapobiega temu za pomocą jawnego wyrównania sekwencji:

```
SELECT setval(pg_get_serial_sequence('teachers', 'id'), coalesce(max(id), 1))
FROM teachers;
SELECT setval(pg_get_serial_sequence('students', 'id'), coalesce(max(id), 1))
FROM students;
SELECT setval(pg_get_serial_sequence('classes', 'id'), coalesce(max(id), 1))
FROM classes;
SELECT setval(pg_get_serial_sequence('activity_logs', 'id'), coalesce(max(id), 1))
FROM activity_logs;
```

Operacje te wykonywane są bezpośrednio po załadowaniu seedów w FastAPI:

```
db.execute(text("SELECT setval(pg_get_serial_sequence('teachers', 'id'),\
coalesce(max(id), 1)) FROM teachers;"))
# ...
db.commit()
```

4. Transakcyjność i Właściwości ACID w Systemie

System opiera swoją spójność i niezawodność na ścisłym przestrzeganiu zasad transakcyjności **ACID** (Atomicity, Consistency, Isolation, Durability) realizowanych przez ORM SQLAlchemy w środowisku wielowątkowym FastAPI.

A. Cykl życia sesji transakcyjnej (Transaction Session Lifecycle)

Wszystkie operacje bazodanowe w backendzie są zorganizowane w myśl wzorca *Unit of Work* (Jednostka pracy) z wykorzystaniem sesji SQLAlchemy (`Session`):

- **Automatyczne rozpoczęcie:** Po nadejściu żądania API, zależność `get_db` inicjuje dedykowaną sesję, która automatycznie rozpoczyna transakcję bazodanową.
- **Jednolity Commit (Zatwierdzenie):** Zmiany wierszy (np. operacje `db.add()` czy `db.delete()`) są zapisywane w pamięci sesji i trwale zatwierdzane w bazie dopiero po wywołaniu `db.commit()`.
- **Bezpieczny Rollback (Wycofanie):** Każdy endpoint modyfikujący dane jest ujęty w blok `try-except`. W razie jakiegokolwiek błędu w walidacji lub komunikacji wywoływane jest `db.rollback()`. Cofa to wszystkie modyfikacje wykonane w danej transakcji i przywraca stan bazy sprzed nadejścia żądania.

B. Atomowość (Atomicity) na przykładzie kaskadowego usuwania

Operacja usunięcia zajęć jest niepodzielna. Wymaga ona modyfikacji wielu tabel:

1. Usunięcie powiązań z tabeli łączącej `enrollments`.
2. Usunięcie rekordu przedmiotu z tabeli `classes`.
3. Dodanie rekordu audytowego w tabeli `activity_logs`.

Wszystkie te trzy kroki zachodzą w ramach **jednej transakcji bazodanowej**. Błąd na dowolnym etapie (np. brak możliwości dodania logu) powoduje wycofanie całej transakcji. Dzięki temu w bazie nie mogą pojawić się "osierocone" zapisy studentów ani błędne logi audytowe bez faktycznie wykonanej akcji.

C. Spójność (Consistency) i Więzy Integralności

Baza danych wymusza spójność schematu poprzez:

- **Klucze obce (Foreign Keys):** Pole `teacher_id` w tabeli `classes` gwarantuje, że zajęcia nie zostaną przypisane do nieistniejącego wykładowcy.
- **Klucze unikalne (Unique Constraints):** Pole `index_number` w tabeli `students` gwarantuje brak możliwości powielenia numeru indeksu na uczelni.

D. Izolacja (Isolation) współbieżnych operacji

W celu zapewnienia najwyższego poziomu izolacji dla krytycznego zasobu, jakim są wolne miejsca na przedmiotach, wdrożono **pesymistyczne blokowanie blokadami zapisu** (`FOR UPDATE`):

- Gdy student zapisuje się na zajęcia, wiersz przedmiotu jest blokowany na poziomie bazy danych.
- Inne transakcje próbujące zapisać się w tym samym ułamku sekundy są kolejgowane na poziomie bazy danych i czekają na zakończenie (`commit/rollback`)

transakcji blokującej. Zapobiega to zjawisku utraconej modyfikacji (Lost Update), gwarantując, że limit miejsc nigdy nie zostanie przekroczony.

5. Możliwości technologiczne

Wybór stosu technologicznego dla systemu ASRPweb nie był przypadkowy i miał na celu zaprezentowanie pełni możliwości nowoczesnych narzędzi do budowy aplikacji sieciowych oraz zarządzania bazami danych.

A. FastAPI – Wysoka wydajność i automatyzacja

Wykorzystanie frameworka FastAPI pozwoliło na zaprezentowanie nowoczesnego podejścia do budowy interfejsów programistycznych (API):

- **Asynchroniczność i wydajność:** Dzięki oparciu o serwer ASGI (Uvicorn), system jest gotowy na obsługę dużej liczby równoległych żądań (np. podczas szczytu logowań na zapisy), co w tradycyjnych frameworkach synchronicznych stanowiłoby problem.
- **Automatyczna generacja dokumentacji:** FastAPI natywnie integruje się ze standardem OpenAPI. W efekcie system generuje interaktywną dokumentację Swagger UI, gotową by testować wszystkie endpointy bez pisania kodu klienckiego.
- **Rygorystyczna walidacja:** Pydantic wymusza ścisłą typizację danych wejściowych. Technologia ta automatycznie odrzuca niepoprawne żądania (np. tekst zamiast liczby miejsc lub błędny format godziny), chroniąc serwer przed błędami jeszcze przed próbą komunikacji z bazą danych.

B. SQLAlchemy – Zaawansowane mapowanie ORM i bezpieczeństwo

Silnik SQLAlchemy został wykorzystany nie tylko jako nakładka na zapytania SQL, ale jako potężne narzędzie architektoniczne:

- **Agnostyczność bazodanowa (Database Agnosticism):** Dzięki warstwie abstrakcji ORM, aplikacja nie jest przywiązana do jednego silnika. Zaprezentowano możliwość płynnego przełączania się między lekką bazą wbudowaną (SQLite) a potężnym silnikiem relacyjnym (PostgreSQL) bez konieczności zmiany ani jednej linijki kodu.
- **Ochrona przed atakami (SQL Injection):** Konstrukcja zapytań obiektowych (np. `db.query(...).filter(...)`) natywnie zapobiega wstrzykiwaniu złośliwego kodu SQL,

dbając o dane wprowadzane przez użytkowników na froncie.

- **Optymalizacja zapytań i leniwe ładowanie (Lazy Loading):** Zamiast pobierać całą bazę do pamięci, technologia pozwala na precyzyjne pobieranie wyłącznie potrzebnych powiązań dzięki złączeniom (JOIN), co drastycznie redukuje zużycie pamięci RAM serwera.

C. Waniliowy JavaScript (Vanilla JS) – Lekkość dla klienta

Po stronie interfejsu zrezygnowano z ciężkich frameworków (np. React, Angular) na rzecz czystego JavaScriptu, Fetch API oraz DOM Manipulation.

- **Demonstracja komunikacji asynchronicznej:** Aplikacja prezentuje, jak w sposób płynny pobierać, edytować i usuwać dane z serwera (CRUD) bez konieczności przeładowywania strony, co zapewnia wrażenia użytkownika charakterystyczne dla nowoczesnych aplikacji typu SPA (Single Page Application).
- **Zarządzanie Stanem Aplikacji (Web Storage API):** Autorski mechanizm bezpiecznego przechowywania stanu (safeStorage oparty na localStorage) pozwala systemowi na "zapamiętanie" wybranej roli użytkownika (Student/Prowadzący/Admin) oraz jego ID pomiędzy przełączaniem się między różnymi podstronami HTML, zachowując ciągłość sesji.
- **Dynamiczna Manipulacja DOM (Document Object Model):** Zamiast renderować gotowe tabele po stronie serwera, JavaScript pobiera z API wyłącznie "surowe" dane w formacie JSON, a następnie za pomocą pętli i pewnych metod buduje skomplikowane struktury HTML (np. karty zajęć, wiersze tabel, dynamiczne listy wyboru ról).
- **Filtrowanie Danych po Stronie Klienta (Client-Side Processing):** Zamiast obciążać serwer każdym wpisaniem znakiem, JavaScript filtruje załadowaną wcześniej tablicę obiektów w pamięci podręcznej przeglądarki (metoda `Array.prototype.filter`), co zapewnia natychmiastowy czas reakcji dla użytkownika wpisującego zapytanie.

6. Instrukcja uruchomienia projektu

1. Ściągnij repozytorium za pomocą `git clone` .
2. Utwórz środowisko dla Python (wersja 3.9+) oraz działający serwer PostgreSQL / gotowość użycia wbudowanego SQLite. Możesz też uruchomić projekt przez Docker.
3. Zainstaluj paczki `pip install fastapi uvicorn sqlalchemy psycopg2-binary pydantic` .
4. Uruchom serwer za pomocą `uvicorn main:app --reload` .

5. Otwórz plik `index.html` przez dowolną przeglądarkę uzyskując dostęp.

Alternatywnie (szybki start): Możesz pominąć kroki 2-4, uruchamiając projekt za pomocą Dockera (`docker-compose up`) i od razu przejść do ostatniego kroku.